

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Kỹ thuật Lập trình - CO1027

Bài tập lớn 2

VUA HẢI TẶC - ONE PIECE
Arc Water 7 - Enies Lobby (phần 2)

Phiên bản 1.0

TP. HỒ CHÍ MINH, THÁNG 05/2026

ĐẶC TẢ BÀI TẬP LỚN

Phiên bản 1.0

1 Chuẩn đầu ra

Sau khi hoàn thành bài tập lớn này, sinh viên ôn lại và sử dụng thành thực:

- Các thao tác đọc/ghi tập tin.
- Con trỏ và cấp phát động.
- Lập trình hướng đối tượng.
- Danh sách liên kết đơn.

2 Dẫn nhập

Trong phần trước, chúng ta đã theo chân băng Mũ Rơm trong hành trình tại Water 7, nơi những mâu thuẫn nội bộ dần được hé lộ và những quyết định quan trọng được đưa ra. Đỉnh điểm của sự kiện là khi Nico Robin bị Chính phủ Thế giới bắt giữ và đưa đến Enies Lobby – một trong ba cơ quan quyền lực lớn nhất của Chính phủ.

Tại Enies Lobby, băng Mũ Rơm buộc phải đối đầu trực diện với lực lượng đặc vụ tình nhuệ CP9 để giải cứu đồng đội của mình. Trận chiến không chỉ đơn thuần là đối đầu giữa các cá nhân, mà còn là sự giằng co giữa thời gian, môi trường chiến trường và các yếu tố chiến thuật khác nhau. Các công trình như Cổng chính, Tòa án, Tháp công lý hay Cầu do dự đều đóng vai trò quan trọng, ảnh hưởng trực tiếp đến tiến trình của cuộc giải cứu.

Trong bài tập lớn này, sinh viên được yêu cầu mô phỏng lại trận chiến tại Enies Lobby thông qua việc xây dựng các lớp (class) và hiện thực các cơ chế tương tác giữa nhân vật, công trình và trạng thái trận chiến. Các nhân vật sẽ hành động theo lượt, thực hiện tấn công hoặc sử dụng kỹ năng đặc biệt, trong khi các công trình liên tục tác động đến diễn biến chung của trận đánh.

3 Các lớp trong chương trình

Bài tập lớn này sử dụng Lập trình Hướng đối tượng để mô tả trận chiến giải cứu Robin của băng Mũ Rơm tại đảo Enies Lobby. Các đối tượng trong Bài tập lớn được biểu diễn thông qua

class và được mô tả như bên dưới.

Lưu ý:

- Sinh viên cần đọc kỹ đề để xác định các lớp kế thừa (derivative class), đề sẽ không mô tả tường minh.
- Sinh viên cần có thể tự xuất thêm các thuộc tính, các phương thức hoặc các class khác để hỗ trợ cho việc hiện thực các class trong Bài tập lớn này.
- Trong khuôn khổ Bài tập lớn này, khi các giá trị tính toán cần làm tròn về số nguyên, sinh viên cần làm tròn lên.
- Trong khuôn khổ Bài tập lớn này, các giá trị `hp`, `energy`, `morale`, `alarmLevel`, `rescueProgress`, `escapeProgress` và `hp` của công trình luôn được làm tròn về khoảng giá trị hợp lệ sau mỗi lần cập nhật. Cụ thể, `hp` của nhân vật nằm trong đoạn từ 0 đến `maxHp`, `hp` của công trình nằm trong đoạn từ 0 đến `maxHP`, `energy`, `morale`, `alarmLevel`, `rescueProgress` và `escapeProgress` nằm trong đoạn từ 0 đến 100. Riêng `busterCallTimer` không được nhỏ hơn 0.

3.1 Nhân vật

Mỗi nhân vật tham gia trận chiến tại Enies Lobby được biểu diễn thông qua một lớp cơ sở trừu tượng **Character**. Đây là lớp cha của tất cả các nhân vật trong chương trình, bao gồm các thành viên băng Mũ Rơm và các đặc vụ CP9.

Yêu cầu: Hiện thực abstract class **Character** với các mô tả sau:

1. Các thuộc tính **protected**:

- `name`: kiểu `string`, tên của nhân vật.
- `hp`: kiểu `int`, lượng máu hiện tại của nhân vật.
- `maxHp`: kiểu `int`, lượng máu tối đa của nhân vật.
- `atk`: kiểu `int`, chỉ số tấn công cơ bản.
- `def`: kiểu `int`, chỉ số phòng thủ cơ bản.
- `speed`: kiểu `int`, tốc độ hành động của nhân vật.
- `energy`: kiểu `int`, năng lượng hiện tại dùng để thi triển kỹ năng.
- `alive`: kiểu `bool`, trạng thái còn khả năng chiến đấu của nhân vật.

2. Constructor **public** với các tham số `name`, `hp`, `atk`, `def`, `speed`, `energy` có ý nghĩa giống với các thuộc tính cùng tên. Constructor gán giá trị của các tham số cho thuộc tính tương

ứng. Giá trị `maxHp` được gán bằng với giá trị `hp`. Nếu `hp` lớn hơn 0 thì `alive` được gán là `true`, ngược lại `alive` được gán là `false`.

```
1 Character(string name, int hp, int atk, int def, int speed, int energy);
```

3. Phương thức hủy ảo với quyền truy cập `public`.

```
1 virtual ~Character();
```

4. Pure virtual method `attack` dùng để thực hiện đòn tấn công thường lên một nhân vật khác. Phương thức nhận vào con trỏ `target` là mục tiêu bị tấn công và tham chiếu `context` là trạng thái hiện tại của trận đánh. Giá trị trả về là lượng sát thương thực tế được tạo ra.

```
1 virtual int attack(Character* target, BattleContext& context) = 0;
```

5. Pure virtual method `specialSkill` dùng để thi triển kỹ năng đặc biệt của nhân vật. Mỗi lớp con sẽ có cách hiện thực kỹ năng khác nhau. Phương thức nhận vào con trỏ `target` là mục tiêu chịu tác động và tham chiếu `context` là trạng thái hiện tại của trận đánh. Giá trị trả về là lượng sát thương được tạo ra.

```
1 virtual int specialSkill(Character* target, BattleContext& context) = 0;
```

6. Virtual method `attack` với tham số là `Building*` dùng để thực hiện đòn tấn công lên công trình.

Phương thức này có hiện thực mặc định không gây sát thương và trả về 0. Các lớp con mô tả nhân vật có khả năng tác động lên công trình bắt buộc override phương thức này.

```
1 virtual int attack(Building* target, BattleContext& context);
```

7. Virtual method `specialSkill` với tham số là `Building*` dùng để thực hiện kỹ năng đặc biệt lên công trình.

Phương thức này có hiện thực mặc định không gây sát thương và trả về 0. Các lớp con mô tả nhân vật có khả năng tác động lên công trình bắt buộc override phương thức này.

```
1 virtual int specialSkill(Building* target, BattleContext& context);
```

8. Virtual method `endTurn` dùng để xử lý các hiệu ứng cuối lượt của nhân vật. Các hiệu ứng này có thể bao gồm hồi năng lượng, cập nhật trạng thái hoặc thay đổi một số thông tin trong `BattleContext`.

```
1 virtual void endTurn(BattleContext& context);
```

9. Method `receiveDamage` dùng để cập nhật máu của nhân vật khi nhận sát thương. Sát thương thực tế nhận vào bằng hiệu của `damage` truyền vào và chỉ số phòng thủ của nhân vật, nếu hiệu này nhỏ hơn 0 thì nhân vật sẽ không nhận sát thương. Nếu sau khi nhận sát thương, `hp` nhỏ hơn hoặc bằng 0 thì `hp` được gán bằng 0 và `alive` được gán là `false`.

```
1 void receiveDamage(int damage);
```

10. Method `isAlive` trả về trạng thái còn khả năng chiến đấu của nhân vật.

```
1 bool isAlive() const;
```

11. Method `getName` trả về tên của nhân vật.

```
1 string getName() const;
```

12. Method `getHP` trả về lượng máu hiện tại của nhân vật.

```
1 int getHP() const;
```

13. Method `getEnergy` trả về lượng năng lượng hiện tại của nhân vật.

```
1 int getEnergy() const;
```

14. Virtual method `isStrawHat` trả về giá trị `true` nếu nhân vật thuộc băng Mũ Rơm, ngược lại trả về `false`. Lớp `StrawHat` cần override phương thức này.

```
1 virtual bool isStrawHat() const;
```

15. Virtual method `isCP9` trả về giá trị `true` nếu nhân vật thuộc lực lượng CP9, ngược lại trả về `false`. Lớp `CP9Agent` cần override phương thức này.

```
1 virtual bool isCP9() const;
```

16. Pure virtual method `str` trả về chuỗi biểu diễn thông tin của nhân vật.

```
1 virtual string str() const = 0;
```

3.2 Băng Mũ Rơm

Băng Mũ Rơm là lực lượng chính trong trận chiến tại Enies Lobby với mục tiêu giải cứu Robin. Các thành viên trong băng đều kế thừa từ lớp `StrawHat`, là lớp con của `Character`.

Yêu cầu: Hiện thực class `StrawHat` với các mô tả sau:

1. Các thuộc tính `protected`:

- `bounty`: kiểu `long long`, tiền truy nã của nhân vật.

2. Constructor (public) nhận các tham số giống lớp `Character`, đồng thời nhận thêm `bounty` để khởi tạo các thuộc tính tương ứng.

```
1 StrawHat(string name, int hp, int atk, int def,  
2         int speed, int energy, long long bounty);
```

3. Phương thức `isStrawHat`:

```
1 virtual bool isStrawHat() const;
```

Trả về giá trị `true` nếu nhân vật thuộc băng Mũ Rơm, ngược lại trả về `false`. Phương thức này được sử dụng để xác định phe của nhân vật trong quá trình xử lý trận đánh.

4. Phương thức `str`:

```
1 virtual string str() const;
```

Trả về chuỗi biểu diễn thông tin của nhân vật thuộc băng Mũ Rơm. Chuỗi trả về có định dạng như sau:

```
1 StrawHat[name=<name>, hp=<hp>, atk=<atk>, def=<def>, speed=<speed>,  
    energy=<energy>, bounty=<bounty>]
```

Các giá trị được in liền nhau, phân tách bằng dấu phẩy và không có khoảng trắng dư thừa ngoài định dạng đã cho.

Ví dụ:

```
1 StrawHat[name=Luffy, hp=120, atk=35, def=20, speed=25, energy=50, bounty  
    =150000000]
```

5. Các phương thức kế thừa từ `Character` sẽ được override ở các lớp con cụ thể.
6. Khi một thành viên của băng hạ gục một đặc vụ CP9, nếu mô tả của nhân vật không nói gì về `morale` thì mặc định giá trị `morale` trong `BattleContext` sẽ tăng thêm 5.

3.2.1 Monkey D. Luffy

Luffy là thuyền trưởng của băng Mũ Rơm, có khả năng chiến đấu mạnh mẽ và càng trở nên nguy hiểm khi bị dồn vào tình huống bất lợi.

Yêu cầu: Hiện thực class `Luffy` kế thừa từ `StrawHat`.

- Phương thức `attack`:

Sát thương gây ra phụ thuộc vào phần trăm máu hiện tại của Luffy. Nếu máu hiện tại lớn hơn 50% máu tối đa, sát thương bằng đúng chỉ số tấn công. Nếu máu lớn hơn 30% và không vượt quá 50%, sát thương tăng thêm 15%. Nếu máu nhỏ hơn hoặc bằng 30%, sát thương tăng thêm 30%.

Nếu đòn tấn công hạ gục mục tiêu, `morale` trong `BattleContext` tăng thêm 5.

- Phương thức `specialSkill`:

Luffy sử dụng Gear Second với chi phí 20 năng lượng. Kỹ năng này gây sát thương bằng 200% chỉ số tấn công, đồng thời tăng tốc độ và sát thương của Luffy thêm 15. Việc sử dụng kỹ năng làm tăng `alarmLevel` thêm 10 và làm giảm máu của Luffy một lượng bằng 8% máu tối đa.

Kỹ năng chỉ được sử dụng khi Luffy còn ít nhất 20 năng lượng và máu không nhỏ hơn 15% máu tối đa.

- Phương thức **endTurn**:

Nếu máu hiện tại nhỏ hơn hoặc bằng 30% máu tối đa, **morale** tăng thêm 3. Nếu Luffy vừa hạ gục một đối thủ trong lượt, Luffy hồi lại 5 năng lượng.

3.2.2 Roronoa Zoro

Zoro là kiếm sĩ của băng Mũ Rơm, có khả năng gây sát thương lớn và dứt điểm mục tiêu hiệu quả.

Yêu cầu: Hiện thực class **Zoro** kế thừa từ **StrawHat**.

- Phương thức **attack**:

Sát thương của Zoro bằng chỉ số tấn công cộng thêm một lượng bằng 20% chỉ số phòng thủ của bản thân. Nếu mục tiêu có lượng máu nhỏ hơn 40% máu tối đa, sát thương tăng thêm 15%.

- Phương thức **specialSkill**:

Zoro sử dụng kỹ năng với chi phí 15 năng lượng, gây sát thương bằng 220% chỉ số tấn công. Nếu mục tiêu có lượng máu nhỏ hơn 50% máu tối đa, sát thương của kỹ năng được tăng thêm 50%. Nếu kỹ năng này hạ gục mục tiêu, Zoro hồi lại 8 năng lượng và **morale** tăng thêm 4.

- Phương thức **endTurn**:

Nếu Zoro hạ gục được một mục tiêu, **morale** tăng lên 6 và chỉ số tấn công của Zoro tăng thêm 5%.

3.2.3 Vinsmoke Sanji

Sanji là một đầu bếp mang phong cách chiến đấu sử dụng cước pháp, có khả năng gây sát thương liên tục và làm suy yếu đối thủ.

Yêu cầu: Hiện thực class **Sanji** kế thừa từ **StrawHat**.

- Phương thức **attack**:

Sát thương bằng chỉ số tấn công cộng thêm một lượng bằng 50% tốc độ hiện tại. Nếu mục tiêu có chỉ số phòng thủ thấp hơn của Sanji, sát thương tăng thêm 10%, quy tắc này không áp dụng đối với công trình.

- Phương thức `specialSkill`:

Sanji sử dụng kỹ năng với chi phí 18 năng lượng, gây sát thương bằng 210% chỉ số tấn công. Sau khi trúng đòn, phòng thủ của mục tiêu giảm 8 đơn vị (bỏ qua nếu là công trình). Nếu mục tiêu là Jabra, phòng thủ giảm 12 đơn vị thay vì 8.

- Phương thức `endTurn`:

Nếu đối thủ bị hạ, `morale` tăng thêm 8 và chỉ số tấn công của Sanji tăng thêm 10%.

3.2.4 Nami

Nami là hoa tiêu của băng, có khả năng kiểm soát chiến trường và gây hiệu ứng bất lợi.

Yêu cầu: Hiện thực class `Nami` kế thừa từ `StrawHat`.

- Phương thức `attack`:

Sát thương bằng chỉ số tấn công, đồng thời bỏ qua 30% phòng thủ của mục tiêu. Nếu là công trình, sát thương chỉ bằng 50% chỉ số tấn công.

- Phương thức `specialSkill`:

Nami sử dụng kỹ năng với chi phí 20 năng lượng, gây sát thương bằng chỉ số tấn công cộng thêm 40. Đồng thời làm giảm tốc độ của mục tiêu 10 đơn vị. Nếu mục tiêu là công trình, sát thương tăng thêm 50%. Sau khi sử dụng, `busterCallTimer` tăng thêm 1 và `alarmLevel` giảm 5.

- Phương thức `endTurn`:

Nếu đối thủ bị hạ, Nami hồi lại 6 năng lượng.

3.2.5 Tony Tony Chopper

Chopper là bác sĩ của băng, vừa có khả năng hồi phục cho đồng đội và vừa có thể chiến đấu.

Yêu cầu: Hiện thực class `Chopper` kế thừa từ `StrawHat`.

- Phương thức `attack`:

Sát thương bằng chỉ số tấn công của Chopper.

- Phương thức `specialSkill`:

Chopper sử dụng kỹ năng hồi máu với chi phí 15 năng lượng, kỹ năng này chỉ có thể được sử dụng lên thành viên của băng Mũ Rơm đang có máu thấp nhất. Kỹ năng này hồi cho mục tiêu một lượng máu bằng 35 cộng thêm 50% chỉ số tấn công của Chopper.

Nếu mục tiêu là Luffy, `morale` trong `BattleContext` tăng thêm 5.

- Phương thức `endTurn`:
Chopper không có hiệu ứng đặc biệt ở cuối lượt.

3.2.6 Usopp (Sogeking)

Do bất hoà trước đó với Luffy, Usopp đã cải trang thành Sogeking trong trận chiến, Sogeking là xạ thủ với khả năng gây hiệu ứng và hỗ trợ chiến thuật.

Yêu cầu: Hiện thực class `Usopp` kế thừa từ `StrawHat`.

- Phương thức `attack`:
Sát thương bằng chỉ số tấn công. Nếu mục tiêu có tốc độ nhỏ hơn 50, sát thương tăng thêm 20%. Nếu mục tiêu là công trình, sát thương chỉ bằng 50% chỉ số tấn công.
- Phương thức `specialSkill`:
Sogeking sử dụng kỹ năng với chi phí 16 năng lượng, gây sát thương bằng 80% chỉ số tấn công. Đồng thời làm giảm tốc độ mục tiêu 12 đơn vị (bỏ qua nếu là công trình). Sau khi sử dụng, `escapeProgress` tăng thêm 8.
- Phương thức `endTurn`:
Sau khi Sogeking tấn công, `morale` toàn đội tăng thêm 10 đơn vị.

3.2.7 Franky (Cutty Flam)

Franky là thủ lĩnh một nhóm côn đồ trên đảo Water Seven, hấn đồng hành cùng đội của Luffy trong quá trình giải cứu Robin. Franky có khả năng gây sát thương lớn và tác động diện rộng.

Yêu cầu: Hiện thực class `Franky` kế thừa từ `StrawHat`.

- Phương thức `attack`:
Sát thương bằng chỉ số tấn công cộng thêm một lượng bằng 30% phòng thủ. Nếu mục tiêu là CP9, sát thương tăng thêm 10%.
- Phương thức `specialSkill`:
Franky có hai kỹ năng:
Kỹ năng Strong Right tiêu tốn 20 năng lượng, gây sát thương bằng 180% chỉ số tấn công và làm giảm tốc độ mục tiêu 8 đơn vị (bỏ qua nếu là công trình). Nếu mục tiêu là Lucci, sát thương tăng thêm 20%.
Kỹ năng Coup de Vent tiêu tốn 30 năng lượng, gây sát thương bằng 120% chỉ số tấn công mục tiêu. Nếu mục tiêu là một `Building`, `Building` đó sẽ ngay lập tức bị phá hủy.

- Phương thức `endTurn`:

Nếu máu hiện tại lớn hơn 70% máu tối đa, phòng thủ của Franky tăng lên 5 đơn vị. Nếu máu hiện tại nhỏ hơn 30% máu tối đa, tấn công của Franky tăng thêm 10%.

3.3 Cipher Pol Number 9 (CP9)

CP9 là lực lượng đặc vụ tinh nhuệ của Chính phủ Thế giới, có nhiệm vụ ngăn chặn băng Mũ Rơm và bảo vệ Enies Lobby. Các thành viên CP9 kế thừa từ lớp `CP9Agent`, là lớp con của `Character`.

Yêu cầu: Hiện thực class `CP9Agent` với các mô tả sau:

1. Các thuộc tính `protected`:

- `doriki`: kiểu `int`, đại diện cho sức mạnh tổng thể của đặc vụ.

2. Constructor nhận các tham số giống lớp `Character`, đồng thời nhận thêm `doriki` để khởi tạo.

```
1 CP9Agent(string name, int hp, int atk, int def,  
2           int speed, int energy, int doriki);
```

3. Phương thức `isCP9`:

```
1 virtual bool isCP9() const;
```

Trả về giá trị `true` nếu nhân vật thuộc lực lượng CP9, ngược lại trả về `false`. Phương thức này được sử dụng để xác định phe của nhân vật trong quá trình xử lý trận đánh.

4. Phương thức `str`:

```
1 virtual string str() const;
```

Trả về chuỗi biểu diễn thông tin của đặc vụ CP9. Chuỗi trả về có định dạng như sau:

```
1 CP9[name=<name>, hp=<hp>, atk=<atk>, def=<def>, speed=<speed>, energy=<  
   energy>, doriki=<doriki>]
```

Các giá trị được in liền nhau, phân tách bằng dấu phẩy và không có khoảng trắng dư thừa ngoài định dạng đã cho. Ví dụ:

```
1 CP9[name=Lucci, hp=150, atk=40, def=25, speed=30, energy=60, doriki  
   =4000]
```

5. Khi một đặc vụ CP9 hạ gục một thành viên băng Mũ Rơm, nếu mô tả của đặc vụ không nói gì về `morale` thì mặc định giá trị `morale` trong `BattleContext` sẽ giảm đi 5.

3.3.1 Rob Lucci

Lucci là đặc vụ mạnh nhất CP9 với sức mạnh áp đảo và khả năng dứt điểm mục tiêu nhanh chóng.

Yêu cầu: Hiện thực class Lucci kế thừa từ CP9Agent.

- Phương thức **attack**:

Sát thương bằng chỉ số tấn công cộng thêm một lượng bằng một phần hai mươi giá trị **doriki**. Nếu mục tiêu có lượng máu nhỏ hơn 50% máu tối đa, sát thương tăng thêm 20%.

- Phương thức **specialSkill**:

Lucci sử dụng kỹ năng với chi phí 25 năng lượng, gây sát thương bằng 280% chỉ số tấn công. Đòn đánh này bỏ qua 50% phòng thủ của mục tiêu. Nếu kỹ năng hạ gục mục tiêu, **morale** giảm thêm 10.

- Phương thức **endTurn**:

Nếu máu hiện tại nhỏ hơn 40% máu tối đa, chỉ số tấn công của Lucci tăng thêm 5%.

3.3.2 Kaku

Kaku là kiếm sĩ với lối tấn công liên hoàn vào nhiều mục tiêu.

Yêu cầu: Hiện thực class Kaku kế thừa từ CP9Agent.

- Phương thức **attack**:

Sát thương bằng chỉ số tấn công.

- Phương thức **specialSkill**:

Kaku sử dụng kỹ năng với chi phí 20 năng lượng, thực hiện ba đòn đánh liên tiếp lên cùng một mục tiêu.

Đòn đầu gây sát thương bằng 120% chỉ số tấn công. Đòn thứ hai gây sát thương bằng 100% chỉ số tấn công. Đòn thứ ba gây sát thương bằng 80% chỉ số tấn công.

Ba đòn đánh này được thực hiện tuần tự. Nếu mục tiêu bị hạ gục trước khi kết thúc ba đòn, các đòn còn lại sẽ không được thực hiện.

- Phương thức **endTurn**:

Kaku không có hiệu ứng đặc biệt ở cuối lượt.

3.3.3 Jabra

Jabra là chiến binh thiên về phòng thủ và phản đòn.

Yêu cầu: Hiện thực class **Jabra** kế thừa từ **CP9Agent**.

- Phương thức **attack**:
Sát thương bằng chỉ số tấn công.
- Phương thức **specialSkill**:
Jabra sử dụng kỹ năng với chi phí 18 năng lượng, gây sát thương bằng 150% chỉ số tấn công.
Nếu máu hiện tại nhỏ hơn 30% máu tối đa, sát thương của kỹ năng tăng thêm 25%. Nếu đòn đánh này hạ gục mục tiêu, **morale** giảm thêm 5.
- Phương thức **endTurn**:
Jabra không có hiệu ứng đặc biệt ở cuối lượt.

3.3.4 Blueno

Blueno có khả năng né tránh và tạo lợi thế trong giao tranh.

Yêu cầu: Hiện thực class **Blueno** kế thừa từ **CP9Agent**.

- Phương thức **attack**:
Sát thương bằng chỉ số tấn công.
- Phương thức **specialSkill**:
Blueno sử dụng kỹ năng với chi phí 15 năng lượng, gây sát thương bằng 130% chỉ số tấn công.
Nếu máu hiện tại của Blueno lớn hơn 50% máu tối đa, sát thương của kỹ năng tăng thêm 20. Nếu máu hiện tại nhỏ hơn hoặc bằng 50% máu tối đa, sát thương tăng thêm 40.
- Phương thức **endTurn**:
Blueno không có hiệu ứng đặc biệt ở cuối lượt.

3.3.5 Kalifa

Kalifa có khả năng làm suy yếu đối thủ và giảm tinh thần chiến đấu.

Yêu cầu: Hiện thực class **Kalifa** kế thừa từ **CP9Agent**.

- Phương thức **attack**:
Sát thương bằng chỉ số tấn công.
- Phương thức **specialSkill**:

Kalifa sử dụng kỹ năng với chi phí 18 năng lượng, gây sát thương bằng 140% chỉ số tấn công. Đồng thời làm giảm **morale** 8 và giảm tốc độ của mục tiêu 6. Nếu mục tiêu là Nami, **morale** giảm 12 thay vì 8.

- Phương thức **endTurn**:
Kalifa không có hiệu ứng đặc biệt ở cuối lượt.

3.3.6 Kumadori

Kumadori có khả năng gây sát thương ổn định và tăng hiệu quả khi bị dồn ép.

Yêu cầu: Hiện thực class **Kumadori** kế thừa từ **CP9Agent**.

- Phương thức **attack**:
Sát thương bằng chỉ số tấn công.
- Phương thức **specialSkill**:
Kumadori sử dụng kỹ năng với chi phí 16 năng lượng, gây sát thương bằng 30 cộng thêm một lượng bằng một phần mười giá trị **doriki**. Nếu máu hiện tại nhỏ hơn 40% máu tối đa, sát thương tăng thêm 25.
- Phương thức **endTurn**:
Kumadori không có hiệu ứng đặc biệt ở cuối lượt.

3.3.7 Fukurou

Fukurou có khả năng theo dõi mục tiêu yếu và gia tăng sát thương.

Yêu cầu: Hiện thực class **Fukurou** kế thừa từ **CP9Agent**.

- Phương thức **attack**:
Sát thương bằng chỉ số tấn công.
- Phương thức **specialSkill**:
Fukurou sử dụng kỹ năng với chi phí 14 năng lượng, gây sát thương bằng 130% chỉ số tấn công. Nếu mục tiêu là nhân vật có lượng máu thấp nhất trong phe đối phương, sát thương tăng thêm 20. Nếu đòn đánh này hạ gục mục tiêu, **morale** giảm thêm 6.
- Phương thức **endTurn**:
Fukurou không có hiệu ứng đặc biệt ở cuối lượt.

3.4 BattleContext

BattleContext lưu trữ trạng thái chung của toàn bộ trận đánh tại Enies Lobby. Các nhân vật và công trình sẽ thông qua lớp này để cập nhật tiến trình và quyết định kết quả trận chiến.

Yêu cầu: Hiện thực class **BattleContext** với các mô tả sau:

1. Các thuộc tính **public**:

- **turnCount**: kiểu **int**, số lượt đã diễn ra.
- **morale**: kiểu **int**, tinh thần của phe Mũ Rơm.
- **alarmLevel**: kiểu **int**, mức báo động tại Enies Lobby.
- **rescueProgress**: kiểu **int**, tiến độ giải cứu Robin.
- **escapeProgress**: kiểu **int**, tiến độ rút lui khỏi Enies Lobby.
- **busterCallTimer**: kiểu **int**, số lượt còn lại trước khi Buster Call xảy ra.
- **mainGateDestroyed**: kiểu **bool**, xác định cổng chính đã bị phá hay chưa. Mặc định là **false**.
- **robinRescued**: kiểu **bool**, xác định Robin đã được giải cứu hay chưa. Mặc định là **false**.
- **bridgeOpened**: kiểu **bool**, xác định cầu thoát hiểm đã được mở hay chưa. Mặc định là **false**.
- **battleEnded**: kiểu **bool**, xác định trận đánh đã kết thúc hay chưa. Mặc định là **false**.
- **resultCode**: kiểu **string**, kết quả cuối cùng của trận đánh.

2. Constructor khởi tạo toàn bộ các giá trị.

3. Phương thức **nextTurn** tăng **turnCount** thêm 1 sau mỗi lượt.

```
1 void nextTurn();
```

3.5 Building

Các công trình tại Enies Lobby được mô hình hóa thông qua lớp trừu tượng **Building**. Các công trình không tham gia trực tiếp vào lượt đánh, nhưng sẽ tác động đến trạng thái trận chiến thông qua **BattleContext**.

Yêu cầu: Hiện thực abstract class **Building** với các mô tả sau:

1. Các thuộc tính **protected**:

- **name**: kiểu `string`, tên công trình.
- **hp**: kiểu `int`, độ bền hiện tại.
- **maxHP**: kiểu `int`, độ bền tối đa.
- **destroyed**: kiểu `bool`, xác định công trình đã bị phá hay chưa.

2. Constructor nhận các tham số **name** và **hp**. Giá trị **maxHP** được gán bằng **hp** ban đầu. Nếu **hp** lớn hơn 0 thì **destroyed** được gán là `false`, ngược lại là `true`.

```
1 Building(string name, int hp);
```

3. Phương thức hủy ảo.

```
1 virtual ~Building();
```

4. Phương thức **receiveDamage** dùng để giảm độ bền của công trình. Nếu sau khi nhận sát thương, **hp** nhỏ hơn hoặc bằng 0, **hp** được gán bằng 0 và **destroyed** được gán là `true`.

```
1 void receiveDamage(int damage);
```

5. Phương thức **isDestroyed** trả về trạng thái của công trình.

```
1 bool isDestroyed() const;
```

6. Pure virtual method **applyEffect** được gọi sau mỗi lượt để áp dụng hiệu ứng của công trình lên **BattleContext**.

```
1 virtual void applyEffect(BattleContext& context) = 0;
```

7. Phương thức **onDestroyed** được gọi khi công trình vừa bị phá hủy để cập nhật trạng thái trong **BattleContext**.

```
1 virtual void onDestroyed(BattleContext& context);
```

8. Phương thức **str**:

```
1 virtual string str() const;
```

Trả về chuỗi biểu diễn thông tin của công trình. Chuỗi trả về có định dạng như sau:

```
1 Building[name=<name>, hp=<hp>, maxHP=<maxHP>, destroyed=<destroyed>]
```

Các giá trị được in liền nhau, phân tách bằng dấu phẩy và không có khoảng trắng dư thừa ngoài định dạng đã cho. Giá trị **destroyed** được in dưới dạng `true` hoặc `false`.

Ví dụ:

```
1 Building[name=MainGate, hp=0, maxHP=200, destroyed=true]
```

3.5.1 MainGate

MainGate là cổng chính của Enies Lobby, là rào cản đầu tiên của băng Mũ Rơm.

Yêu cầu: Hiện thực class **MainGate** kế thừa từ **Building**.

- Phương thức `applyEffect`:
Nếu `MainGate` chưa bị phá, tiến độ giải cứu Robin không tăng trong lượt này.
- Phương thức `onDestroyed`:
Khi `MainGate` bị phá, `mainGateDestroyed` được gán là `true`, `rescueProgress` tăng thêm 20 và `morale` tăng thêm 5.

3.5.2 Courthouse

Courthouse là khu vực trung tâm làm tăng mức báo động.

Yêu cầu: Hiện thực class `Courthouse` kế thừa từ `Building`.

- Phương thức `applyEffect`:
Nếu Courthouse chưa bị phá, `alarmLevel` tăng thêm 5 sau mỗi lượt.
- Phương thức `onDestroyed`:
Khi bị phá, `alarmLevel` giảm 20 và không còn tăng trong các lượt tiếp theo.

3.5.3 TowerOfJustice

TowerOfJustice là nơi giam giữ Robin.

Yêu cầu: Hiện thực class `TowerOfJustice` kế thừa từ `Building`.

- Phương thức `applyEffect`:
Nếu `mainGateDestroyed` là `true` và Robin chưa được cứu, `rescueProgress` tăng thêm 5 sau mỗi lượt.
Nếu `rescueProgress` lớn hơn hoặc bằng 100, `robinRescued` được gán là `true` và `morale` tăng thêm 10.

3.5.4 BridgeOfHesitation

BridgeOfHesitation là con đường rút lui khỏi Enies Lobby.

Yêu cầu: Hiện thực class `BridgeOfHesitation` kế thừa từ `Building`.

- Phương thức `applyEffect`:
Nếu `robinRescued` là `true`, `bridgeOpened` được gán là `true` và `escapeProgress` tăng thêm 5 mỗi lượt.
Nếu `escapeProgress` lớn hơn hoặc bằng 100, trận đánh kết thúc với kết quả `STRAW_HAT_WIN`.

3.5.5 BusterCallShip

BusterCallShip đại diện cho áp lực thời gian của trận đánh.

Yêu cầu: Hiện thực class `BusterCallShip` kế thừa từ `Building`.

- Phương thức `applyEffect`:
Nếu công trình chưa bị phá, `busterCallTimer` giảm 1 sau mỗi lượt.
Nếu `busterCallTimer` nhỏ hơn hoặc bằng 0, trận đánh kết thúc với kết quả `BUSTER_CALL`.
- Phương thức `onDestroyed`:
Khi công trình bị phá, `busterCallTimer` tăng thêm 3.

3.6 EniesLobbyBattle

Lớp `EniesLobbyBattle` là lớp quản lý toàn bộ trận đánh tại Enies Lobby. Lớp này chịu trách nhiệm đọc dữ liệu khởi tạo, quản lý danh sách nhân vật, quản lý các công trình, điều phối lượt đánh, cập nhật trạng thái trong `BattleContext` và xác định kết quả cuối cùng của trận chiến.

Yêu cầu: Hiện thực class `EniesLobbyBattle` với các mô tả sau:

1. Các thuộc tính `private`:

- `strawHats`: kiểu `Character**`, mảng động lưu các nhân vật thuộc băng Mũ Rơm. Số lượng nhân vật của băng Mũ Rơm không quá 7.
- `strawHatCount`: kiểu `int`, số lượng nhân vật băng Mũ Rơm hiện có.
- `cp9Agents`: kiểu `Character**`, mảng động lưu các đặc vụ CP9. Số lượng đặc vụ không quá 7.
- `cp9Count`: kiểu `int`, số lượng đặc vụ CP9 hiện có.
- `buildings`: kiểu `Building**`, mảng động lưu các công trình trên chiến trường. Số lượng công trình không quá 5.
- `buildingCount`: kiểu `int`, số lượng công trình hiện có.
- `turnOrder`: danh sách liên kết đơn quản lý thứ tự hành động của các nhân vật.
- `context`: kiểu `BattleContext`, trạng thái hiện tại của trận đánh.
- `maxTurns`: kiểu `int`, số lượt tối đa của trận đánh.

2. Constructor nhận vào đường dẫn đến tập tin cấu hình. Constructor này gọi phương thức `loadFromFile` để đọc dữ liệu và khởi tạo trận đánh.

```
1 EniesLobbyBattle(const string& filename);
```

3. Destructor có nhiệm vụ giải phóng toàn bộ vùng nhớ đã cấp phát động cho các nhân vật, công trình, mảng động và danh sách liên kết.

```
1 ~EniesLobbyBattle();
```

4. Phương thức `loadFromFile` đọc dữ liệu từ tập tin có tên `filename` và khởi tạo toàn bộ trận đánh.

```
1 void loadFromFile(const string& filename);
```

Tập tin đầu vào gồm nhiều dòng, mỗi dòng mô tả một nhóm dữ liệu. Các dòng có thể xuất hiện theo thứ tự bất kỳ, nhưng phải tuân theo một trong các định dạng sau:

```
1 CONTEXT morale alarmLevel rescueProgress escapeProgress busterCallTimer  
    maxTurns  
2 STRAW_HAT name hp atk def speed energy bounty  
3 CP9 name hp atk def speed energy doriki  
4 BUILDING name hp
```

Trong đó, dòng `CONTEXT` dùng để khởi tạo các giá trị ban đầu của `BattleContext` và `maxTurns`. Dòng `STRAW_HAT` dùng để tạo nhân vật thuộc băng Mũ Rơm. Dòng `CP9` dùng để tạo đặc vụ CP9. Dòng `BUILDING` dùng để tạo công trình trên chiến trường.

Sau khi đọc xong dữ liệu hợp lệ, phương thức phải xây dựng `turnOrder` bằng cách gọi `buildTurnOrder`.

5. Phương thức `addStrawHat` thêm một nhân vật băng Mũ Rơm vào mảng động `strawHats`

```
1 void addStrawHat(Character* character);
```

6. Phương thức `addCP9Agent` thêm một đặc vụ CP9 vào mảng động `cp9Agents`.

```
1 void addCP9Agent(Character* character);
```

7. Phương thức `addBuilding` thêm một công trình vào mảng động `buildings`.

```
1 void addBuilding(Building* building);
```

8. Phương thức `buildTurnOrder` xây dựng danh sách lượt đánh. Danh sách này chỉ chứa các con trỏ kiểu `Character*`; các công trình không nằm trong danh sách lượt đánh.

```
1 void buildTurnOrder();
```

Thứ tự ban đầu của `turnOrder` được tạo theo quy tắc: tất cả nhân vật băng Mũ Rơm theo thứ tự đọc từ file được đưa vào trước, sau đó đến tất cả đặc vụ CP9 theo thứ tự đọc từ file.

Mỗi node trong danh sách liên kết chứa một con trỏ `Character*` và một con trỏ đến node kế tiếp.

9. Phương thức `runBattle` thực hiện toàn bộ trận đánh.

```
1 void runBattle();
```

Trận đánh được xử lý theo từng lượt cho đến khi `battleEnded` bằng `true` hoặc số lượt đạt `maxTurns`. Quy trình xử lý mỗi lượt được thực hiện theo các bước sau:

- (a) Lấy nhân vật đầu tiên trong `turnOrder`.
- (b) Nếu nhân vật đã bị hạ, bỏ qua lượt của nhân vật này và chuyển sang bước tiếp theo.
- (c) Nếu nhân vật còn sống:
 - Xác định mục tiêu theo đúng quy tắc chọn mục tiêu.
 - Nếu nhân vật còn đủ năng lượng, thực hiện `specialSkill`.
 - Ngược lại, thực hiện `attack`.
 - Sau khi hành động kết thúc, gọi `endTurn`.
- (d) Đưa node hiện tại xuống cuối danh sách `turnOrder`.
- (e) Gọi `processBuildings` để áp dụng hiệu ứng của tất cả công trình.
- (f) Tăng `turnCount` thêm 1.
- (g) Gọi `checkEndCondition` để kiểm tra trạng thái trận đánh.

Nếu số lượt đạt `maxTurns` mà trận đánh chưa kết thúc, `battleEnded` được gán là `true` và `resultCode` được gán là `TIME_OUT`.

10. Phương thức `processTurn` xử lý lượt hành động của một nhân vật.

```
1 void processTurn(Character* character);
```

Nếu `character` không còn sống thì không thực hiện hành động. Nếu `character` thuộc băng Mũ Rơm, mục tiêu được chọn theo quy tắc chọn mục tiêu của Straw Hat. Nếu `character` thuộc CP9, mục tiêu được chọn theo quy tắc chọn mục tiêu của CP9.

Nếu mục tiêu là nhân vật, sát thương được áp dụng lên mục tiêu thông qua `receiveDamage`.

Nếu mục tiêu là công trình, sát thương được áp dụng lên công trình thông qua `receiveDamage`; nếu công trình vừa bị phá, gọi `onDestroyed` của công trình đó.

11. Phương thức `processBuildings` gọi hiệu ứng của toàn bộ công trình sau mỗi lượt hành động.

```
1 void processBuildings();
```

Với mỗi công trình trong mảng `buildings`, gọi phương thức `applyEffect`. Nếu công trình đã bị phá, hiệu ứng chính của công trình đó không còn được áp dụng.

12. Phương thức `checkEndCondition` kiểm tra điều kiện kết thúc trận đánh.

```
1 void checkEndCondition();
```

Các điều kiện kết thúc trận đánh được kiểm tra theo thứ tự sau:

Thứ tự	Điều kiện	Kết quả
1	<code>robinRescued = true</code> và <code>escapeProgress</code> <code>>= 100</code>	STRAW_HAT_WIN
2	<code>busterCallTimer <= 0</code>	BUSTER_CALL
3	Toàn bộ nhân vật trong <code>strawHats</code> đã bị hạ	CP9_WIN
4	Toàn bộ nhân vật trong <code>cp9Agents</code> đã bị hạ	STRAW_HAT_WIN_BY_DEFEAT_CP9
5	<code>turnCount >= maxTurns</code>	TIME_OUT

13. Phương thức `getResult` trả về chuỗi kết quả sau khi trận đánh kết thúc.

```
1 string getResult() const;
```

Chuỗi kết quả có định dạng:

```
1 resultCode turnCount morale alarmLevel rescueProgress escapeProgress  
  busterCallTimer
```

Ví dụ:

```
1 STRAW_HAT_WIN 32 78 25 100 100 4
```

Các quy tắc của trận chiến:

- Quy tắc chọn mục tiêu của băng Mũ Rơm. Trong lượt của một nhân vật thuộc băng Mũ Rơm, mục tiêu được chọn theo thứ tự ưu tiên sau:
 - Nếu `mainGate` chưa bị phá, mục tiêu là `MainGate`.
 - Nếu `MainGate` đã bị phá, `alarmLevel` lớn hơn hoặc bằng 50 và `Courthouse` chưa bị phá, mục tiêu là `Courthouse`.
 - Nếu hai điều kiện trên không thỏa, `busterCallTimer` nhỏ hơn hoặc bằng 5 và `BusterCallShip` chưa bị phá, mục tiêu là `BusterCallShip`.
 - Nếu các điều kiện trên không thỏa và Robin chưa được giải cứu, mục tiêu là đặc vụ CP9 còn sống đầu tiên trong mảng `cp9Agents`.
 - Nếu Robin đã được cứu, mục tiêu là `BridgeOfHesitation`. Trong trường hợp này, hành động của nhân vật có thể được xem là hỗ trợ mở đường rút lui; nếu công trình này đã bị phá hoặc không tồn tại, nhân vật sẽ tấn công CP9 còn sống đầu tiên.
 - Nếu nhân vật hiện tại là Chopper và đủ năng lượng dùng `specialSkill`, mục tiêu là thành viên băng Mũ Rơm còn sống có hp thấp nhất.
- Quy tắc chọn mục tiêu của CP9: trong lượt của một đặc vụ CP9, mục tiêu luôn là thành viên băng Mũ Rơm còn sống đầu tiên trong mảng `strawHats`. CP9 không tấn công công trình.

3. Quy tắc sử dụng kỹ năng:

- Ở lượt của một nhân vật, nếu nhân vật còn đủ năng lượng để sử dụng `specialSkill`, nhân vật sẽ dùng `specialSkill`. Nếu không đủ năng lượng, nhân vật sẽ dùng `attack`.
- Chi phí năng lượng của từng kỹ năng được mô tả trong phần lớp nhân vật cụ thể. Sau khi dùng kỹ năng, năng lượng của nhân vật phải giảm đúng bằng chi phí tương ứng. Nếu không đủ năng lượng, kỹ năng không được thực hiện.

4. Quy tắc tấn công nhân vật:

- Khi mục tiêu là một nhân vật, sát thương do `attack` hoặc `specialSkill` tạo ra được áp dụng lên mục tiêu. Nếu sau khi nhận sát thương, máu của mục tiêu nhỏ hơn hoặc bằng 0, máu của mục tiêu được gán bằng 0 và mục tiêu được xem là bị hạ.
- Nếu người tấn công thuộc băng Mũ Rơm và hạ gục mục tiêu, `morale` tăng thêm 5. Nếu người tấn công thuộc CP9 và hạ gục mục tiêu, `morale` giảm 5.

4 Yêu cầu

Để hoàn thành bài tập lớn này, sinh viên phải:

1. Đọc toàn bộ tập tin mô tả này.
2. Tải xuống tập tin `initial.zip` và giải nén nó. Sau khi giải nén, sinh viên sẽ nhận được các tập tin: `main.cpp`, `main.h`, `eniesLobby.h`, `eniesLobby.cpp`, và các file dữ liệu đọc mẫu. Sinh viên phải nộp 2 tập tin là `eniesLobby.h` và `eniesLobby.cpp` nên không được sửa đổi tập tin `main.h` khi chạy thử chương trình.
3. Sinh viên sử dụng câu lệnh sau để biên dịch:

```
g++ -o main main.cpp eniesLobby.cpp -I . -std=c++11
```

Sinh viên sử dụng câu lệnh sau để chạy chương trình:

```
./main
```

Các câu lệnh trên được dùng trong command prompt/terminal để biên dịch và chạy chương trình. Nếu sinh viên dùng IDE để chạy chương trình, sinh viên cần chú ý: thêm đầy đủ các tập tin vào project/workspace của IDE; thay đổi lệnh biên dịch của IDE cho phù hợp. IDE thường cung cấp các nút (button) cho việc biên dịch (Build) và chạy chương trình (Run). Khi nhấn Build IDE sẽ chạy một câu lệnh biên dịch tương ứng, thông thường câu lệnh chỉ biên dịch file `main.cpp`. Sinh viên cần tìm cách cấu hình trên IDE để thay đổi lệnh biên dịch: thêm file `eniesLobby.cpp`, thêm option `-std=c++11, -I .`

4. Chương trình sẽ được chấm trên nền tảng Unix. Nền tảng chấm và trình biên dịch của sinh viên có thể khác với nơi chấm thực tế. Nơi nộp bài trên LMS được cài đặt để giống với nơi chấm thực tế. Sinh viên phải chạy thử chương trình trên nơi nộp bài và phải sửa tất cả các lỗi xảy ra ở nơi nộp bài LMS để có đúng kết quả khi chấm thực tế.
5. Sửa đổi các file `eniesLobby.h`, `eniesLobby.cpp` để hoàn thành bài tập lớn này và đảm bảo hai yêu cầu sau:
 - Chỉ có 1 lệnh **include** trong tập tin `eniesLobby.h` là **#include "main.h"** và một include trong tập tin `eniesLobby.cpp` là **#include "eniesLobby.h"**. Ngoài ra, không cho phép có một **#include** nào khác trong các tập tin này.
 - Hiện thực các hàm được mô tả ở các Nhiệm vụ trong BTL này.
 - Hoàn thiện các class để đảm bảo các tính chất cơ bản của lập trình hướng đối tượng
6. Sinh viên được khuyến khích viết thêm các class và các hàm để hoàn thành BTL này.
7. Sinh viên bắt buộc phải cung cấp phần hiện thực cho tất cả các class và phương thức được liệt kê trong BTL này. Nếu không thì SV sẽ bị 0 điểm. Nếu đó là class hoặc phương thức mà SV chưa thể giải quyết được, thì SV phải cung cấp phần hiện thực rỗng chỉ gồm cặp dấu mở đóng ngoặc nhọn `{}`.

5 Nộp bài

Sinh viên chỉ nộp 2 tập tin: `eniesLobby.h` và `eniesLobby.cpp`, trước thời hạn được đưa ra trong đường dẫn "Assignment 2 - Submission". Có một số testcase đơn giản được sử dụng để kiểm tra bài làm của sinh viên nhằm đảm bảo rằng kết quả của sinh viên có thể biên dịch và chạy được. Sinh viên có thể nộp bài bao nhiêu lần tùy ý nhưng chỉ có bài nộp cuối cùng được tính điểm. Vì hệ thống không thể chịu tải khi quá nhiều sinh viên nộp bài cùng một lúc, vì vậy sinh viên nên nộp bài càng sớm càng tốt. Sinh viên sẽ tự chịu rủi ro nếu nộp bài sát hạn chót. Khi quá thời hạn nộp bài, hệ thống sẽ đóng nên sinh viên sẽ không thể nộp nữa. Bài nộp qua các phương thức khác đều không được chấp nhận.

6 Một số quy định khác

- Sinh viên phải tự mình hoàn thành bài tập lớn này và phải ngăn không cho người khác đánh cắp kết quả của mình. Nếu không, sinh viên sẽ bị xử lý theo quy định của trường vì gian lận.
- Mọi quyết định của giảng viên phụ trách bài tập lớn là quyết định cuối cùng.

- Sinh viên không được cung cấp testcase sau khi chấm bài, sinh viên sẽ được cung cấp phân bố điểm của BTL.

7 Gian lận

Bài tập lớn phải được sinh viên TỰ LÀM. Sinh viên sẽ bị coi là gian lận nếu:

- Có sự giống nhau bất thường giữa mã nguồn của các bài nộp. Trong trường hợp này, TẤT CẢ các bài nộp đều bị coi là gian lận. Do vậy sinh viên phải bảo vệ mã nguồn bài tập lớn của mình.
- Bài của sinh viên bị sinh viên khác nộp lên.
- Sinh viên không hiểu mã nguồn do chính mình viết, trừ những phần mã được cung cấp sẵn trong chương trình khởi tạo. Sinh viên có thể tham khảo từ bất kỳ nguồn tài liệu nào, tuy nhiên phải đảm bảo rằng mình hiểu rõ ý nghĩa của tất cả những dòng lệnh mà mình viết. Trong trường hợp không hiểu rõ mã nguồn của nơi mình tham khảo, sinh viên được đặc biệt cảnh báo là KHÔNG ĐƯỢC sử dụng mã nguồn này; thay vào đó nên sử dụng những gì đã được học để viết chương trình.
- Nộp nhầm bài của sinh viên khác trên tài khoản cá nhân của mình.
- Sử dụng mã nguồn từ các công cụ có khả năng tạo ra mã nguồn mà không hiểu ý nghĩa.

Trong trường hợp bị kết luận là gian lận, sinh viên sẽ bị điểm 0 cho toàn bộ môn học (không chỉ bài tập lớn).

KHÔNG CHẤP NHẬN GIẢI THÍCH NÀO VÀ KHÔNG CÓ NGOẠI LỆ!

Sau mỗi bài tập lớn được nộp, sẽ có một số sinh viên được gọi phỏng vấn ngẫu nhiên để chứng minh rằng bài tập lớn vừa được nộp là do chính mình làm.

8 Kết luận

Trận chiến tại Enies Lobby đánh dấu một trong những khoảnh khắc quan trọng nhất của băng Mũ Rơm, khi họ sẵn sàng đối đầu trực diện với Chính phủ Thế giới để giải cứu Nico Robin. Từ việc phá vỡ công chính, tiến vào tòa án, cho đến cuộc đối đầu với các đặc vụ CP9 và cuộc chạy đua với thời gian trước khi Buster Call được kích hoạt, toàn bộ diễn biến đã tạo nên một chuỗi sự kiện căng thẳng và đầy kịch tính.

Trong bài tập này, sinh viên được tái hiện lại những diễn biến đó thông qua việc xây dựng các đối tượng và quy tắc tương tác trong chương trình, qua đó hình dung rõ hơn cách một câu chuyện có thể được mô hình hóa thành một hệ thống có cấu trúc.



CHÚC CÁC BẠN GIẢI CỨU THÀNH CÔNG ROBIN!